

Expert Decision Replay Platform

Complete Project Documentation

Prepared by: Vedansh Mudgal

Infosys Springboard Virtual Internship 7.0

Personal Learning Project — Full Stack Implementation

This document is the complete technical reference for the Expert Decision Replay Platform: architecture, database design, every module built, the API surface, and the reasoning behind key design decisions made throughout development.

Table of Contents

1. Project Overview
2. Technology Stack
3. System Architecture
4. Database Design & ER Diagram
5. Module-by-Module Breakdown
6. Authentication & Security
7. Role-Based Access Control
8. Complete API Reference
9. Frontend Design System
10. Key Design Decisions & Trade-offs
11. Known Limitations & Future Work
12. Setup & Local Installation

1. Project Overview

The Expert Decision Replay Platform is a centralized system for recording significant organizational decisions — the problem, the alternatives considered, who reviewed and approved it, and the final outcome. The goal is to preserve institutional knowledge so teams can learn from past decisions rather than repeating mistakes.

This document covers the personal learning-track implementation, which extends beyond the original project brief in several areas (role-ordered multi-level approvals, automatic version history, audit logging with soft-deletes, and a distinctive visual identity) to deepen full-stack engineering practice.

2. Technology Stack

Layer	Technology	Purpose
Backend framework	FastAPI (Python)	REST API, automatic interactive docs (Swagger UI)
Database	PostgreSQL	Relational data storage with enforced foreign keys
ORM	SQLAlchemy	Python classes mapped to database tables
Migrations	Alembic	Versioned, reversible schema change tracking
Auth	JWT (python-jose) + bcrypt	Stateless token authentication, one-way password hashing
PDF generation	ReportLab	In-memory PDF report generation, streamed to the client
Excel generation	OpenPyXL	In-memory .xlsx spreadsheet generation
Frontend framework	React + Vite	Component-based UI, fast local dev server
Routing	React Router	Client-side page navigation without full reloads
HTTP client	Axios	API requests, with interceptors for auth + centralized error handling
Styling	Custom CSS	Bespoke "case file" visual identity (no UI framework)

3. System Architecture

The backend follows a layered, router-based structure. Rather than one large file containing every endpoint, related endpoints are grouped into dedicated router modules, each responsible for one feature area:

- **main.py** — application entrypoint; creates the FastAPI app, configures CORS, and registers every router.
- **database.py** — builds the database engine/session using a safely-encoded connection URL (handles special characters in passwords).
- **models.py** — every SQLAlchemy table definition; the single source of truth for the database schema.
- **schemas.py** — Pydantic models defining exact API request/response shapes, deliberately separate from the database models (e.g. password hashes are never exposed, even if the database row contains them).
- **auth.py** — password hashing, JWT creation/verification, and the `get_current_user / require_admin` dependencies used across every protected route.
- **helpers.py** — shared cross-cutting logic used by multiple routers: `get_next_required_role` (approval ordering), `log_action` (audit logging), `notify` (notifications), `create_decision_version` (version snapshots), `build_team_detail`.
- **routers/** — one file per feature area: `auth`, `users`, `teams`, `decisions`, `alternatives`, `attachments`, `comments`, `approvals`, `admin`, `notifications`, `audit`.

The frontend mirrors this separation: **pages/** hold full-screen views, **components/** hold reusable pieces shared across pages (`AppHeader`, `RoleStamp`, `StatusStamp`, `NotificationBell`), and **services/api.js** centralizes every backend call behind a single configured Axios instance.

Request Flow Example — Approving a Decision

- Frontend sends the outcome (Approved/Rejected/Escalated) with the stored JWT attached automatically by an Axios request interceptor.
- Backend's `get_current_user` verifies the token and identifies the reviewer.
- `get_next_required_role` checks the Approval history to confirm it is genuinely this role's turn — otherwise the request is rejected with 403.
- A new Approval row is saved; the parent Decision's status is recalculated based on how many levels have now signed off.
- A version snapshot, an audit log entry, and notifications (to the creator and the next reviewer) are all created as part of the same transaction.
- The updated decision, approval history, and notification state are returned and reflected immediately in the UI.

4. Database Design & ER Diagram

The schema centers on the Decision entity, with every other feature (Alternatives, Attachments, Comments, Approvals, Version History) attached to it via a foreign key. Users and Teams form a secondary, mutually-referencing pair (a Team has a manager who is a User; a User belongs to a Team) supporting the organizational structure.

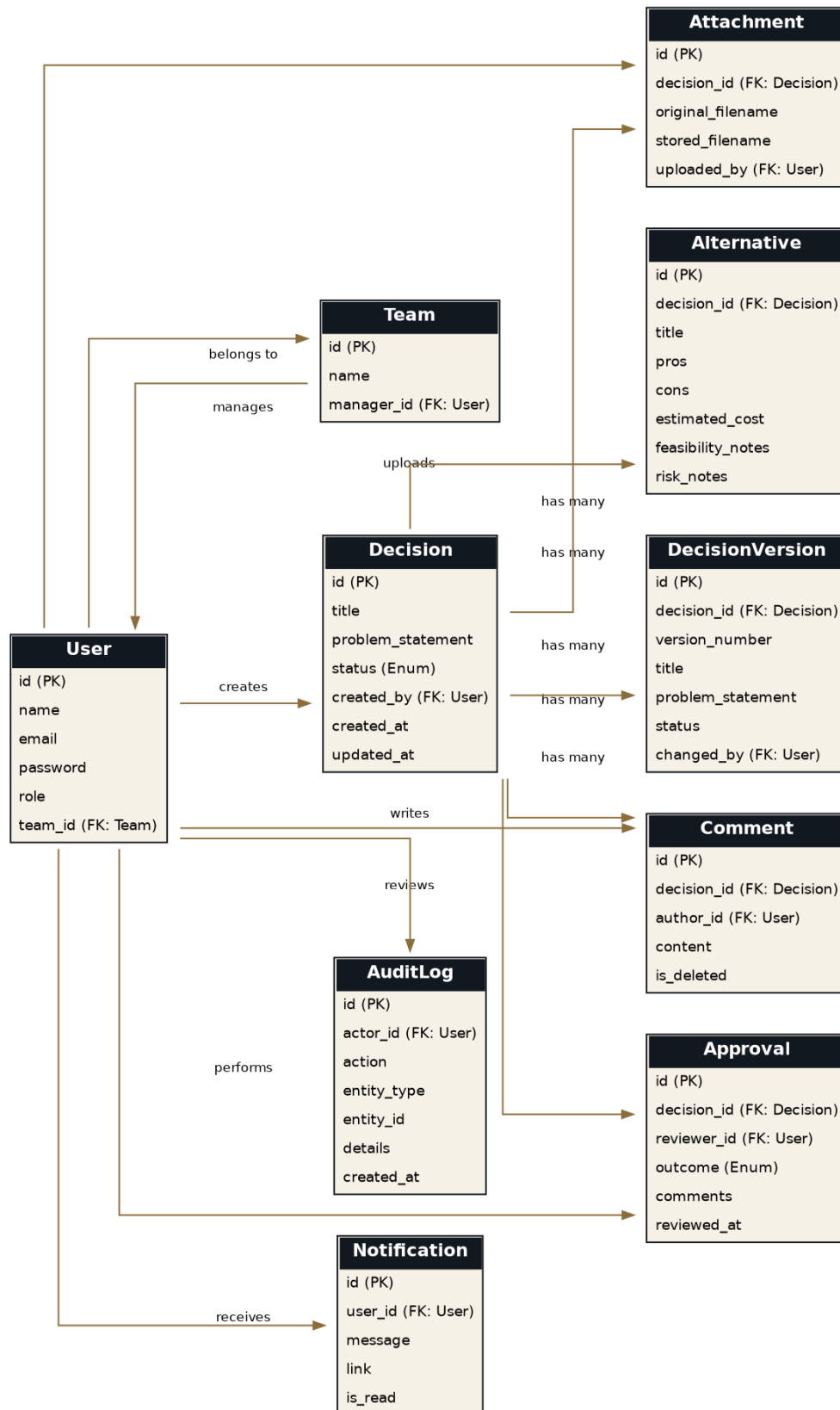


Figure 1: Entity-Relationship Diagram — all 10 tables and their relationships.

Table Summary

Table	Purpose
User	Accounts, credentials (hashed), role, optional team membership
Team	Groups of users, with one designated manager
Decision	The core record: title, problem statement, status, creator
Alternative	Options considered for a decision, with pros/cons/cost/risk
Attachment	Uploaded files linked to a decision (metadata only; files live on disk)
Comment	Discussion thread on a decision (supports soft-delete)
Approval	One review event in the multi-level approval chain
DecisionVersion	A frozen snapshot of a decision's state at a point in time
AuditLog	A record of who did what, to what, and when, system-wide
Notification	A message queued for a specific user about a relevant event

5. Module-by-Module Breakdown

5.1 Authentication & User Management

Registration, login, JWT issuance, and profile retrieval. Passwords are hashed with bcrypt; new accounts always default to the Employee role.

5.2 Team Management

Admins create teams and assign managers. Managers can add/remove members only for the team they manage, verified via a per-resource permission check (`team.manager_id == current_user.id`), not just a blanket role check.

5.3 Decision Management

Full CRUD on decisions, with search/filter support, permission-gated editing (creator or Admin only), and automatic version snapshots taken before every edit.

5.4 Alternative Analysis

Multiple options can be recorded per decision, each with independent pros, cons, cost estimate, feasibility notes, and risk notes, displayed as comparison cards.

5.5 Multi-Level Approval Workflow

Decisions are reviewed in a strict order — Reviewer, then Manager, then Administrator. The system tracks exactly how many levels have signed off and blocks out-of-order or duplicate reviews. A rejection at any level immediately finalizes the decision as Rejected.

5.6 Discussion & Comments

Threaded comments per decision. Deletion is a soft-delete (`is_deleted` flag) rather than a hard delete, preserving the record for audit purposes while hiding the content from the visible thread.

5.7 Attachments

Files are uploaded with a randomly generated storage filename (avoiding collisions) while preserving the original filename for display. Downloads are authenticated via a token-bearing request rather than a plain link, since browsers cannot attach custom headers to direct link clicks.

5.8 Audit Logging

A single `log_action()` helper is called from every meaningful mutation across the system. It does not commit independently — it is added to the same transaction as the action it is logging, guaranteeing the log and the action succeed or fail together.

5.9 Notifications

In-app notifications generated by a shared `notify()` helper, triggered by approvals and comments. The frontend bell polls for an unread count every 30 seconds and lazily loads the full list only when opened.

5.10 Reporting & Export

PDF (single decision) and Excel (all decisions) exports are generated entirely in memory using ReportLab and OpenPyXL respectively, and streamed directly to the client — no export file is ever written to the server's disk.

6. Authentication & Security Details

- **Password storage:** bcrypt hashing, one-way — the application can never recover or display a real password.
- **Session mechanism:** stateless JWT — the server verifies a token's signature and expiry without needing a database lookup for every request.
- **Secrets management:** database credentials and the JWT signing key are stored in a local `.env` file, excluded from version control; only a placeholder `.env.example` is committed.
- **Frontend token handling:** a shared Axios response interceptor automatically clears the token and redirects to Login on any 401 response, and attaches a human-readable `friendlyMessage` to every error for consistent UI display.
- **Least privilege:** endpoints expose only the narrowest data required for a task (e.g. a dedicated `/users/unassigned` endpoint for Managers, rather than granting broader access to the full user list).

7. Role-Based Access Control

Role	Can Do
Employee	Create decisions, view all decisions, comment, add alternatives, upload attachments
Reviewer	Everything an Employee can, plus first-level approval review
Manager	Everything a Reviewer can, plus second-level approval review and managing their own team
Administrator	Everything, plus: manage all teams, change any user's role, view Audit Log, view system-wide stats

8. Complete API Reference

Auth & Users

Method	Endpoint	Access	Purpose
POST	/users	Public	Register a new user (Employee role)
POST	/login	Public	Log in, returns JWT
GET	/users/me	Any user	Get own profile
GET	/users	Admin	List all users
GET	/users/unassigned	Manager/Admin	List users not on a team
PATCH	/users/{id}/role	Admin	Change a user's role
PATCH	/users/{id}/team	Manager (own team)/Admin	Assign user to team
DELETE	/users/{id}/team	Manager (own team)/Admin	Remove user from team

Teams

Method	Endpoint	Access	Purpose
POST	/teams	Admin	Create a team
GET	/teams	Any user	List all teams
GET	/teams/mine	Any user	Get own team
GET	/teams/{id}	Any user	Get a specific team's detail
PATCH	/teams/{id}	Admin	Rename team / assign manager
DELETE	/teams/{id}	Admin	Delete a team

Decisions

Method	Endpoint	Access	Purpose
POST	/decisions	Any user	Create a decision
GET	/decisions	Any user	List/search/filter decisions
GET	/decisions/mine	Any user	List own decisions
GET	/decisions/pending-review	Reviewer/Manager/Admin	List decisions awaiting this user's review
GET	/decisions/export/excel	Any user	Export all decisions as .xlsx
GET	/decisions/{id}	Any user	Get one decision
PATCH	/decisions/{id}	Creator/Admin	Edit a decision (creates a version snapshot)
DELETE	/decisions/{id}	Any user*	Delete a decision

GET	/decisions/{id}/versions	Any user	Get version history
GET	/decisions/{id}/export/pdf	Any user	Export one decision as PDF

*Delete currently has no ownership check — noted as a known limitation, see Section 11.

Alternatives, Attachments, Comments, Approvals

Method	Endpoint	Access	Purpose
POST	/decisions/{id}/alternatives	Any user	Add an alternative
GET	/decisions/{id}/alternatives	Any user	List alternatives
POST	/decisions/{id}/attachments	Any user	Upload a file
GET	/decisions/{id}/attachments	Any user	List attachments
GET	/attachments/{id}/download	Any user	Download a file
DELETE	/attachments/{id}	Uploader/Admin	Delete a file
POST	/decisions/{id}/comments	Any user	Post a comment
GET	/decisions/{id}/comments	Any user	List comments
DELETE	/comments/{id}	Author/Admin	Soft-delete a comment
POST	/decisions/{id}/approvals	Correct role only	Submit a review outcome
GET	/decisions/{id}/approvals	Any user	List approval history

Admin, Notifications, Audit

Method	Endpoint	Access	Purpose
GET	/admin/stats	Admin	System-wide counts and status breakdown
GET	/audit-logs	Admin	200 most recent system actions
GET	/notifications	Any user	List own notifications
GET	/notifications/unread-count	Any user	Get unread count
PATCH	/notifications/{id}/read	Owner only	Mark one notification read
PATCH	/notifications/mark-all-read	Any user	Mark all own notifications read

9. Frontend Design System

Rather than a generic dashboard aesthetic, the interface adopts a deliberate "case file" visual language, reflecting the platform's purpose of recording and reviewing decisions.

- **Palette:** dark navy/charcoal page background (representing a desk), warm cream "paper" surfaces for actual content cards, brass/gold accents, and muted red/green for rejected/approved states.
- **Typography:** Lora (serif) for headings — a documentary, official-record feel; IBM Plex Mono for labels, statuses, and metadata — evoking case numbers and filing codes; Inter for body text and form inputs.
- **Signature motif:** role and status "stamps" — tilted, bordered labels resembling a rubber stamp pressed onto a physical file, used consistently for both user roles and decision statuses.
- **Reusable components:** AppHeader (consistent navigation across every page), RoleStamp / StatusStamp (color-coded badges via a lookup-table pattern), a shared forms.css for all inputs/textareas.

10. Key Design Decisions & Trade-offs

- **One team per user** — a user has a single team_id, not a many-to-many relationship. Simpler to reason about and matches the report's scope; would require a join table if multi-team membership were ever needed.
- **Soft-delete for comments only, not decisions/attachments** — comments are conversational history worth preserving for audit purposes; attachments/decisions use hard deletes for simplicity, since their deletion is a rarer, more deliberate action.
- **Router-based backend structure** — the codebase was deliberately refactored midway through development from a single main.py into feature-based router modules, improving readability as the project grew past ~10 endpoints.
- **Frontend permission mirroring** — the frontend recalculates "whose turn is it to approve" independently of the backend for UX purposes only; the backend's check remains the actual source of truth and cannot be bypassed by frontend logic.
- **In-memory export generation** — PDF/Excel files are streamed directly from memory rather than written to disk, avoiding orphaned export files accumulating on the server over time.

11. Known Limitations & Future Work

- Decision deletion currently has no ownership/permission check — any logged-in user can delete any decision. Flagged for a fix (restrict to creator or Admin, matching the edit permission pattern).
- Multi-team membership is not supported (see Trade-offs above).
- Admin Dashboard summary statistics are not yet clickable/drillable into full lists — planned enhancement.
- No automated test suite yet; testing has been manual and exploratory throughout development.
- Not yet containerized (Docker) — planned for the final project phase.
- Notification delivery is in-app only; no email notifications.

12. Setup & Local Installation

Backend

cd edrp-backend → python -m venv venv → activate venv → pip install -r requirements.txt → copy .env.example to .env and fill in real values → alembic upgrade head → uvicorn main:app --reload

Frontend

cd edrp-frontend → npm install → npm run dev

Interactive API documentation is available at <http://127.0.0.1:8000/docs> once the backend is running.

End of Full Project Documentation.